

Feature Engineering

Convertir el mundo en columnas sin hacer trampa

Enrique Bolaños Abner Xocop

25 de agosto de 2026

Veritas · Libertas · Jvstitia

Agenda

- 1.** Los modelos comen números
- 2.** Variables categóricas
- 3.** Escalamiento
- 4.** Fechas, texto e interacciones
- 5.** Leakage
- 6.** Importancia de features
- 7.** Ejercicios
- 8.** Cierre

Todo tiene que volverse número

Un modelo no ve “Quetzaltenango” ni “martes” ni “camisa talla L”. Ve una matriz X de números y nada más.

Feature engineering es esa traducción, y es donde se gana o se pierde la mayoría de los proyectos:

- Un modelo mediocre con buenos features gana casi siempre
- Un modelo sofisticado con features malos no tiene nada que hacer
- Las decisiones de esta sesión son **irreversibles** para el modelo: lo que no esté en X , no existe

Los datos ponen el techo; el modelo solo trata de alcanzarlo

One-hot y ordinal

```
1 import pandas as pd
2 df = pd.DataFrame({"ciudad": ["GT", "QZ", "GT"],
3                        "talla": ["S", "L", "M"]})
4 pd.get_dummies(df["ciudad"], prefix="ciudad")
5 # Ordinal: SOLO cuando el orden es real
6 orden = {"S": 0, "M": 1, "L": 2}
7 df["talla_num"] = df["talla"].map(orden)
```

Codificar ciudad como GT= 0, QZ= 1, ESC= 2 le dice al modelo que Escuintla está el doble de lejos que Quetzaltenango, lo cual es falso y el modelo se lo cree. Ese es el error de encoding más común y el más silencioso.

Alta cardinalidad

One-hot sobre una columna con 3 categorías da 3 columnas. Sobre municipio (340 en Guatemala) da 340, casi todas de ceros.

- La matriz explota y muchas columnas tienen 5 filas en 1
- Los árboles se ahogan buscando splits en columnas casi vacías
- Categorías nuevas en producción no existen en el encoder

Opciones razonables: agrupar en niveles superiores (departamento), quedarse con el top- k y mandar el resto a OTROS, o codificar por una propiedad de la categoría (población, ingreso medio, distancia a la capital).

En `OneHotEncoder`, `handle_unknown="ignore"` evita que una categoría nueva reviente el pipeline en producción.

Target encoding: poderoso y peligroso

Reemplazar cada categoría por el **promedio del target** en esa categoría. Una sola columna, muchísima señal.

Municipio	Tasa de mora	Feature
Mixco	0.12	0.12
Villa Nueva	0.19	0.19
Antigua	0.07	0.07

Por qué es peligroso

El feature contiene el target. Si el promedio se calcula con todo el dataset, cada fila mira su propia respuesta: el modelo se ve perfecto en validación y fracasa en producción. Se calcula **solo en train**, y conviene suavizar hacia la media global cuando la categoría tiene pocas filas.

¿Por qué escalar?

Dos features en unidades distintas no compiten en igualdad de condiciones:

Feature	Rango	Peso en la distancia
Edad (años)	18 a 70	despreciable
Ingreso (Q)	3,000 a 90,000	todo

- **Distancias** (k-NN, k-Means, SVM): la variable de rango grande decide sola quién está cerca
- **Gradientes** (regresión, redes): escalas dispares hacen que el descenso zigzaguee y converja lento
- **Regularización**: penalizar $\sum \beta_j^2$ es injusto si los β_j están en unidades distintas

StandardScaler, MinMaxScaler y quién no los necesita

```
1 from sklearn.preprocessing import StandardScaler
2 from sklearn.preprocessing import MinMaxScaler
3 sc = StandardScaler()      #  $(x - \text{media}) / \text{std} \rightarrow \text{media } 0$ 
4 mm = MinMaxScaler()        #  $(x - \text{min}) / (\text{max} - \text{min}) \rightarrow [0, 1]$ 
5 X_tr_s = sc.fit_transform(X_tr)
6 X_te_s = sc.transform(X_te) # transform, NO fit_transform
```

- StandardScaler: el default razonable; no acota el rango
- MinMaxScaler: útil cuando se necesita $[0, 1]$, pero un outlier aplasta a todos los demás contra el piso
- **Los árboles no lo necesitan:** parten por umbrales (“ingreso $> 12,000$ ”), y ese orden no cambia al escalar

Fechas: una fecha son muchos features

```
1 import numpy as np
2 s = pd.to_datetime(["2026-08-25", "2026-09-15"])
3 s.dayofweek          # 1, 1  (martes)
4 s.month, s.day       # (8, 9), (25, 15)
5 es_finde = s.dayofweek >= 5
6 # Ciclicas: la hora 23 y la 0 estan pegadas
7 hora = np.array([23, 0, 1])
8 sen = np.sin(2*np.pi*hora/24)
9 cos = np.cos(2*np.pi*hora/24)
```

Un timestamp crudo es casi inútil. Lo útil es el día de la semana, el mes, si es feriado, si es quincena, cuántos días faltan para el evento. Codificar la hora como 0 a 23 le dice al modelo que las 23:00 y la 00:00 están lejísimos; el par seno/coseno arregla exactamente eso.

Binning, texto e interacciones

```
1 # Binning: continuo -> categorias con sentido de negocio
2 pd.cut(df["edad"], bins=[0, 25, 40, 60, 120],
3        labels=["joven", "adulto", "maduro", "mayor"])
4 # Texto: bag-of-words (sesion de Naive Bayes)
5 from sklearn.feature_extraction.text import CountVectorizer
6 # Interacciones: lo que importa es el producto
7 df["precio_por_m2"] = df["precio"] / df["area"]
```

El binning pierde información, así que solo se justifica cuando la relación es escalonada o el negocio ya piensa en esas categorías. Las **interacciones** (razones, productos) son donde el conocimiento del dominio le gana a cualquier modelo: `precio_por_m2` vale más que `precio` y `area` por separado.

La fuga: información que no vas a tener

Leakage es entrenar con información que no estará disponible al momento de predecir. El síntoma es siempre el mismo: *el modelo salió demasiado bueno*.

Dos familias:

- **Del futuro:** predecir si un paciente será hospitalizado usando la columna `días_de_hospitalización`. En producción esa columna todavía no existe
- **Del target:** cualquier feature construido mirando la respuesta, incluyendo las respuestas del conjunto de prueba

Un 0.99 de accuracy no es una buena noticia: es una alarma

Los dos casos clásicos

```
1 # MAL: el scaler ve la media y std de TODO
2 X_s = StandardScaler().fit_transform(X)
3 X_tr, X_te = train_test_split(X_s)
4 # MAL: target encoding con todo el dataset
5 df["muni_enc"] = df.groupby("muni")["y"].transform("mean")
6 # BIEN: partir primero, ajustar despues
7 X_tr, X_te = train_test_split(X)
8 sc = StandardScaler().fit(X_tr)
9 X_tr, X_te = sc.transform(X_tr), sc.transform(X_te)
```

En el caso del scaler la fuga es sutil (solo se filtran media y desviación) y el optimismo es pequeño. En el target encoding la fuga es brutal y puede regalar 20 puntos de accuracy imaginarios.

La regla: fit solo en train

Todo lo que **aprenda algo de los datos** se ajusta en train. Los Pipelines de scikit-learn lo garantizan por construcción:

```
1 from sklearn.pipeline import Pipeline
2 from sklearn.linear_model import LinearRegression
3 pipe = Pipeline([("sc", StandardScaler()),
4                  ("modelo", LinearRegression())])
5 pipe.fit(X_tr, y_tr)      # el scaler ve solo train
6 pipe.score(X_te, y_te)
```

MÁS ALLÁ DEL CÓDIGO

El 17 de septiembre veremos que el Pipeline es además la única forma correcta de hacer validación cruzada.

¿Cuáles features sirven?

```
1 # 1) Correlacion con el target: barata y miope
2 df.corr(numeric_only=True)["y"].sort_values()
3 # 2) Coeficientes de un modelo lineal ESCALADO
4 modelo.coef_
5 # 3) Importancia de un arbol (sesion del 10 de sep)
6 modelo.feature_importances_
```

- La correlación solo ve relaciones lineales y una variable a la vez
- Los coeficientes solo son comparables si los features están en la **misma escala**
- Dos features muy correlacionados se reparten la importancia y ambos parecen inútiles

Importancia no es causalidad. Guarden esa frase para el 29 de septiembre.

Ejercicio 1: encodear un dataset mixto

```
1 import seaborn as sns
2 df = sns.load_dataset("tips")
3 # a) one-hot para day y time
4 # ¿Que va aqui?
5 # b) ordinal para size (el orden SI es real)
6 # c) un feature de interaccion: propina por persona
7 # d) ¿cuantas columnas quedaron al final?
```

Ejercicio 2: provocar un leakage

Midan el optimismo que produce escalar antes de partir.

```
1 from sklearn.model_selection import train_test_split
2 # a) VERSION CON FUGA: escalar X completo y luego partir
3 # ¿Que va aqui? Entrenen y midan en test
4 # b) VERSION LIMPIA: partir, fit en train, transform
5 # c) resten los dos scores: ese es el optimismo
6 # Prueben tambien con target encoding: la fuga es mayor
```

Ejercicio 3: el pipeline correcto

```
1 from sklearn.pipeline import Pipeline
2 from sklearn.compose import ColumnTransformer
3 # a) ColumnTransformer: OneHot a categoricas,
4 #     StandardScaler a numericas
5 pre = ...
6 # b) Pipeline: pre + un modelo
7 pipe = ...
8 # c) fit en train, score en test. Comparen con el (a)
9 #     del ejercicio anterior
```

Resolvemos los tres en el notebook de la sesión; lo que falte queda de práctica para la casa.

Lo esencial de hoy

- Categóricas: **one-hot** por defecto, ordinal solo si el orden existe, target encoding con mucho cuidado
- **Escalar** importa para distancias y gradientes; los árboles no lo necesitan
- Una fecha son muchos features; las cíclicas piden seno y coseno
- Las **interacciones** del dominio le ganan a los modelos
- **Leakage**: si el resultado es demasiado bueno, búsquenlo
- La regla que resume todo: **fit solo en train**, y Pipeline para no olvidarlo

Próxima clase (jueves 27): k-NN y Perceptrón

Hoy se entrega la **HDT 3**